

内存虚拟化、地址翻译与页生命周期

从连续重定位、分页、TLB、缺页异常到 COW、Swap、mmap 与物理页分配

408 专题手册 · 操作系统 × 计算机组成原理桥梁篇

核心模型：虚拟内存是一套地址意义管理系统

程序只产生虚拟地址；操作系统定义这些地址代表什么，
硬件高速执行翻译；异常路径再把控制权交回内核。

分析主线：

Virtual Region → Mapping → Physical Residency
→ Translation → Access / Fault

其中必须始终区分四件事：

Allocation ≠ Mapping ≠ Residency ≠ Translation

一个虚拟地址范围可以已经“属于进程”，却尚未有物理页；一个页面可以曾经驻留、后来被换出；一个映射可以合法存在，但当前访问权限仍不允许本次操作。

术语预备：地址、页和驻留状态

- **虚拟地址 (VA)**：程序指令产生的地址；它先经过当前进程的地址翻译，不能直接当作 RAM 的物理位置。
- **物理地址 (PA)**：内存控制器实际访问的 RAM 地址；同一个 VA 在不同进程中可以对应不同 PA。
- **页 (page) 与页框 (frame)**：虚拟地址空间按固定大小切成页，物理内存按同样大小切成页框；页是虚拟对象，页框是物理承载位置。
- **页表项 (PTE)**：记录一个虚拟页如何翻译及其权限、有效位、脏位等状态的表项。
- **TLB**：Translation Lookaside Buffer，缓存近期的 VA 页号到 PA 页框号及权限，减少每次访问都遍历页表的成本。
- **VMA**：Virtual Memory Area，描述一段连续虚拟地址范围的归属、权限和后备对象；它不等于已经分配的物理页。
- **缺页异常 (Page Fault)**：访问的页没有可直接使用的有效映射，或权限不满足，CPU 因而转入内核处理；它不等于“文件不存在”。
- **驻留 (Residency)**：某页当前是否有物理页框承载；**后备对象 (Backing Object)**：页不在 RAM 时可从中恢复数据的文件、Swap 或零填充来源。

本册所有“分配、映射、驻留、翻译”判断都分别回答不同问题，不能用其中一个词替代另外三个。

系统的通用推理：缺页中断（Page Fault）中的按序约束

在虚拟内存处理缺页异常时，同样遵循 OS 通用系统推理引擎（OS System Reasoning Engine）：

State (PTE/Frame/Disk) → Page Fault (缺页异常)

→ Failure (未装载读垃圾/竞态)

→ Invariant (PTE.valid = 1 $\implies$ Frame 数据就绪)

因此内核处理必须施加**按序约束（Ordering Constraint）**：必须先完成磁盘 Page-In 填入 Frame，最后才将 PTE.valid 置 1 并更新 TLB。若顺序颠倒，就会在并发下产生脏读与未定义行为。

目录

1 第一原则：先把五种“内存问题”拆开	3
1.1 先消除“分配”一词的层次混淆	3
1.2 本册统一五层模型	3
2 为什么必须虚拟化地址：四个矛盾	4
2.1 重定位：程序不应该知道自己被放在 RAM 的哪里	4
2.2 隔离：不同进程可以使用“相同地址”，却不应访问同一内存	4
2.3 共享：隔离不是“不许共享”，而是共享必须显式受控	4
2.4 稀疏与按需：只为实际访问的页面分配物理内存	4
3 从连续重定位到分页：为什么方案一步步升级	5
3.1 静态重定位：在装入时一次性改地址	5
3.2 动态重定位：把“地址意义”推迟到运行时	5
3.3 连续分配与空闲分区管理：分配、切割、回收与合并	5
3.4 分段 (Segmentation)：表达程序的逻辑模块结构	6
3.5 段页式 (Segmented Paging)：兼顾逻辑结构与物理利用率	6
3.6 分页的关键跨越：固定粒度 + 非连续物理放置	7
4 分页数学模型：地址为什么要拆成 VPN 与 Offset	7
4.1 页内偏移保持不变	7
4.2 单级页表为什么会太大	7
5 多级页表：用稀疏数据结构描述稀疏地址空间	8
5.1 核心思想不是“多查几次”，而是“未使用的区域不建低级表”	8
5.2 PTE 不只是 PFN	8
6 TLB：翻译本身也需要缓存	9

6.1	为什么页表不能每次都完整查	9
6.2	快路径与慢路径	9
6.3	三种 miss/fault 必须彻底分开	9
6.4	有效访问时间 EAT 的模型意识	9
7	Page Fault: 虚拟内存的慢路径入口	10
7.1	定义: 不是错误, 而是“硬件无法完成当前翻译/访问”	10
7.2	统一处理流程	10
7.3	Page Fault 的六种常见原因与处理	11
8	一个 Page 的一生: 从不存在到驻留、变脏、被回收	11
8.1	Page Lifecycle 是虚拟内存最重要的动态模型	11
8.2	Resident 与 Valid 是两个容易混淆的词	12
9	Demand Paging: 为什么“需要时再做”通常更划算	12
9.1	Lazy 是一种通用系统策略	12
9.2	代价: 首次访问被推入慢路径	12
10	页面置换: RAM 不够时谁应该留下	12
10.1	先问为什么需要 replacement	12
10.2	缺页率 (Page Fault Rate) 的多因素控制模型	12
10.3	四个经典策略的思想, 而不是口号	13
10.4	Dirty Page 为什么影响置换代价	14
10.5	Frame Allocation 与 Replacement Scope	14
10.6	最小页框数与有效驻留集: 正确性底线不等于性能目标	14
10.7	Working Set 与 PFF: 一个预测模型, 一个反馈控制器	14
11	Working Set 与 Thrashing: 负载过高时越调度越慢	15
11.1	局部性为什么支持虚拟内存	15
11.2	Thrashing: 页框不足导致反复换入换出	15
12	物理内存管理: Buddy 应该放在这里, 而不是和 malloc 混在一起	16
12.1	历史连续分区与 Free-Space Management	16
12.2	Buddy System: 管理成组连续物理页	16
12.3	为什么还需要 Slab/SLUB 一类对象分配器	16
13	共享、Copy-on-Write 与 fork: 首次写入时才复制物理页	16
13.1	fork 的语义承诺	16
13.2	COW 的完整状态转换	16

13.3 为什么 fork + exec 特别受益	17
14 mmap：把文件页直接映射进虚拟地址空间	17
14.1 先分 anonymous 与 file-backed	17
14.2 mmap 的核心不是“把整个文件读进内存”	17
14.3 标准 read 与 mmap 的结构对比	18
15 综合题一：一次虚拟地址访问	18
16 综合题二：一个页面的生命周期	19
17 综合题三：fork + COW 的状态变化	19
18 最小调用接口：先定位地址、映射、驻留还是翻译	19
19 教材模型与现代工程：哪些细节要分层记忆	20
20 跨科桥梁：计组与 OS 在一次访存中怎样接力	20
21 六条核心结论	21
参考资料与工程边界	21

1 第一原则：先把五种“内存问题”拆开

1.1 先消除“分配”一词的层次混淆

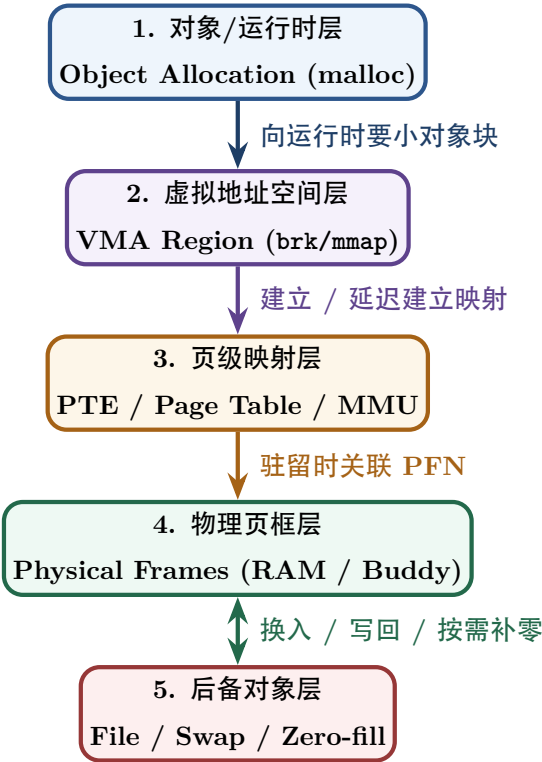
同一个“分配”一词，至少可能指五件完全不同的事：

内存概念层次	管理的对象与单位	核心解决的核心问题	典型代表机制 / 数据结构
语言/运行时对象分配	Object / Chunk	malloc(100) 从哪里得到一块可用小块？	Heap Allocator、Free List、Size Class
进程虚拟地址空间	VA Range / VMA	哪些虚拟地址属于本进程？权限与属性是什么？	VMA (vm_area_struct)、brk、mmap
页级地址映射	VPN → PFN	某个虚拟页号 (VPN) 当前映射到哪个物理页框？	多级页表 (Page Table) / 页表项 (PTE)
物理内存框分配	Physical Frame (PFN)	内核如何从全局空闲物理页中分配连续页？	Free Page List、Buddy System 伙伴系统
驻留与页面回收	Resident Page	主存紧张时，谁保留在 RAM、谁写回/淘汰？	Page Reclaim、Page Replacement、Swap/Writeback

最容易形成的错误模型

malloc() 并不等于“每次调用系统调用，请 OS 用 First Fit 找一块物理内存”。现代用户态分配器通常先管理已经取得的虚拟地址区域；只有需要扩展 arena 或建立新映射时，才通过 brk/mmap 等接口向内核请求地址空间。即使地址空间已经扩大，也可能等到首次访问时才获得物理页。

1.2 本册统一五层模型



每层回答不同问题

- VMA: 这个 VA 是否属于进程? 允许 R/W/X 吗? 背后是什么对象?
- PTE: 这个虚拟页当前怎样翻译? 权限和驻留状态是什么?
- Physical Frame: 真实 RAM 中哪一页现在承载数据?
- Backing Object: 如果页不在 RAM, 数据从哪里恢复? 文件、Swap, 还是直接补零?

2 为什么必须虚拟化地址: 四个矛盾

2.1 重定位: 程序不应该知道自己被放在 RAM 的哪里

最早的核心问题可以写成:

$$\text{Program Address} \neq \text{Physical Placement.}$$

若程序内部写死物理地址, 那么每次换装入位置都必须修改程序; 运行中移动、交换、共享更困难。

最简单的动态重定位模型是:

$$PA = Base + VA, \quad 0 \leq VA < Limit.$$

硬件完成加法与越界检查, OS 在切换进程时配置 Base/Limit。

2.2 隔离: 不同进程可以使用“相同地址”, 却不应访问同一内存

例如两个进程都可以访问自己的:

$$VA = 0x400000.$$

通过不同映射:

$$VPN_A \rightarrow PFN_{12}, \quad VPN_B \rightarrow PFN_{91},$$

它们的地址数值相同, 物理对象却完全不同。

2.3 共享: 隔离不是“不许共享”, 而是共享必须显式受控

若只读共享库页需要被多个进程使用, 可以让:

$$VPN_A \rightarrow PFN_{37}, \quad VPN_B \rightarrow PFN_{37}$$

并设置只读/可执行权限。于是:

$$\boxed{\text{独立虚拟地址空间} \not\Rightarrow \text{所有物理页都独占}}$$

2.4 稀疏与按需: 只为实际访问的页面分配物理内存

虚拟地址空间允许大量“洞”和尚未驻留的区域。程序可以先取得一个大虚拟范围, 仅在实际访问时再分配物理页:

$$\text{Reserve VA} \rightarrow \text{First Touch} \rightarrow \text{Page Fault} \rightarrow \text{Allocate Frame.}$$

虚拟内存的三个最重要目标

从系统设计角度，虚拟内存同时追求：

1. **Transparency**：程序以为自己拥有简单、连续的地址空间；
2. **Protection**：不同主体不能越权访问；
3. **Efficiency**：常见地址访问必须靠硬件快路径完成，而不能每次进入内核。

3 从连续重定位到分页：为什么方案一步步升级

3.1 静态重定位：在装入时一次性改地址

经典静态重定位在 load time 修改需要重定位的位置，使程序以后直接使用最终地址。优点是执行期硬件简单；缺点是运行后移动困难，灵活性和隔离能力有限。

3.2 动态重定位：把“地址意义”推迟到运行时

动态重定位把逻辑地址保留下来，直到每次访问时由 MMU 翻译。最简单实现是 Base/Limit。

408 辨析：静态/动态“内存分配”与静态/动态“重定位”不是一对概念

语言层面的静态变量、栈变量、heap 动态分配，讨论的是**对象何时取得存储空间**；静态/动态重定位讨论的是**程序地址在什么时候变成物理地址**。两组术语不能混成同一维度。

3.3 连续分配与空闲分区管理：分配、切割、回收与合并

如果进程必须占用一段连续物理内存区，系统必须记录空闲分区表/链。空闲区状态转换链：

Free Block $\xrightarrow{\text{Allocation}}$ Split (Allocated + Remainder) $\xrightarrow{\text{Recycle}}$ Merge (Adjacent Free Blocks)

常用的适配算法（First Fit, Best Fit, Worst Fit, Next Fit）只是不同的找块策略。连续分配无法避免在分区之间产生微小、不连续的物理空闲区，即**外部碎片 (External Fragmentation)**：

$$\sum Free_i \geq Request \quad \text{但} \quad \max Free_i < Request.$$

连续分区的完整状态机

连续分配题要把“分配”和“回收”分开落笔。单一连续分配只有一个用户区；固定分区在系统启动时预划定，进程填不满的部分是**内部碎片**；动态分区按请求切割，内部碎片很少但会产生外部碎片。动态分区的空闲表/链至少记录首址、大小与状态；分配后切成“已分配块 + 余块”，回收时必须按地址判断前后邻接并合并。四种回收情况的表项变化为：只邻前改前项大小、只邻后把首址改为回收区首址、前后都邻则三块合并并删除后一项、两边都不邻则新建表项。总空闲量相同并不代表可满足同一个大请求，判据是最大连续空闲块而非空闲量总和。

紧凑 (compaction) 只有在进程可动态重定位时才成立；它搬移的是已分配进程而不是空闲块本身，代价是大量复制与停顿，因此只能缓解外部碎片，不能消除固定分区的内部碎片。

Overlay vs Swapping 核心辨析

Overlay ≠ Swapping

- **Overlay（覆盖）**：发生在同一个程序内部，程序员/编译器划分逻辑模块，让互斥模块共享同一块内存区。主要是**程序逻辑组织**问题。
- **Swapping（对换）**：由 OS 自动控制，将暂时不运行的整个进程或页面换出到外存对换区。主要是**内存资源管理**问题。

3.4 分段 (Segmentation): 表达程序的逻辑模块结构

分页成功消除了外部碎片，但强制把内存按固定大小打碎，破坏了程序的自然逻辑结构。分段按逻辑模块（代码段、数据段、堆栈段）划分子空间：

对比维度	分页机制 (Paging)	分段机制 (Segmentation)
划分驱动因素	系统物理管理驱动（固定大小 Page/Frame）	程序员/编译器逻辑驱动（可变大小 Segment）
主要解决问题	消灭外部碎片，提高物理内存利用率	天然表达逻辑结构，方便段级共享与保护
地址空间维度	一维（对程序员透明，线性虚拟地址）	二维（显式指定 Segment No + Offset）
碎片类型	存在页内 ** 内部碎片 (Internal Fragmentation)**	存在段间 ** 外部碎片 (External Fragmentation)**
段/页表结构	PTE 包含 PFN + Permission	段表包含 Segment Base + Limit + Access Rights

段表字段结构与地址翻译 段表是将逻辑段号映射到实际物理地址的入口，每个段表项包含：

Segment Table Entry = Segment Base | Segment Limit | Present | Access Rights | Dirty | Swap Addr

翻译三步走：

1. 越界检查：校验 $Offset < Segment\ Limit$ ；若否，触发段越界异常（Segmentation Fault）；
2. 权限校验：校验 Access Rights（如尝试写入只读代码段则触发权限异常）；
3. 地址生成： $PA = Segment\ Base + Offset$ 。

3.5 段页式 (Segmented Paging): 兼顾逻辑结构与物理利用率

段页式内存管理融合了分段与分页的优势：

Logical Segment (User View) $\xrightarrow{\text{Segment Table}}$ Page Table (Logical Pages) $\xrightarrow{\text{Page Table}}$ Physical Frames (MMU View)

逻辑地址格式为：Segment No | Page No | Page Offset。

- 对程序员：呈现逻辑分段，方便按段共享代码与设置独立权限；
- 对 MMU：段内按物理页离散分配，彻底消除外部碎片！

3.6 分页的关键跨越：固定粒度 + 非连续物理放置

分页把：

- 虚拟地址空间切成 page；
- 物理内存切成同样大小的 frame；
- 通过 page table 建立任意 VPN → PFN 映射。

因此进程物理页可以散落在 RAM 中，而虚拟空间依旧看起来连续。

4 分页数学模型：地址为什么要拆成 VPN 与 Offset

4.1 页内偏移保持不变

若页大小为：

$$P = 2^p \text{ Bytes,}$$

则虚拟地址可以写成：

$$\boxed{VA = VPN \mid Offset}, \quad |Offset| = p.$$

页表只翻译页号：

$$VPN \rightarrow PFN,$$

而页内 offset 不变：

$$\boxed{PA = PFN \mid Offset}.$$

地址题第一反应

1. 先由 page size 得 offset 位数；
2. 再用虚拟地址总位数减去 offset 得 VPN 位数；
3. 若多级页表，再按每级索引位数继续拆 VPN；
4. 最后不要漏掉 offset 原样拼回物理地址。

页大小与位数推导

页越大，页表项数量和 TLB 覆盖压力通常下降，顺序局部性可能更好，但页内碎片、一次缺页的调入量和无效预取风险上升；页越小则相反。给定地址宽度 v 、页大小 2^p 和 PTE 大小 e ，页数为 2^{v-p} 、单级页表大小为 $2^{v-p}e$ 。多级页表把 VPN 再分为各级索引；每级索引位数由该级页表页能容纳的表项数决定，而不是凭感觉平均切分。

4.2 单级页表为什么会太大

若虚拟地址 v 位、页大小 2^p ，虚拟页数：

$$N_{page} = 2^{v-p}.$$

若每个 PTE 占 e Bytes，则单级页表大小：

$$S_{PT} = 2^{v-p} \cdot e.$$

即使实际只用很少地址，也必须为整个 VPN 空间准备表项，因此出现多级页表。

5 多级页表：用稀疏数据结构描述稀疏地址空间

5.1 核心思想不是“多查几次”，而是“未使用的区域不建低级表”

一个多级地址可以抽象成：

$$VA = I_1 | I_2 | \cdots | I_k | Offset.$$

每一级索引选择下一张表，只有实际使用的虚拟区域才需要逐层建立低级页表页。

多级页表：用树形索引描述稀疏地址空间

多级页表拿更多查询步骤换更少的页表空间。它与文件系统的 inode 间接块、目录树、B+ 树一样，都是“稀疏大命名空间”的层次化描述方法。

5.2 PTE 不只是 PFN

应把 PTE 看成：

$$PTE = Mapping + Permission + State.$$

典型信息包括：

- 物理页框号或下一层页表地址；
- present/valid；
- user/supervisor；
- read/write/execute；
- accessed/reference；
- dirty；
- 架构或 OS 自定义状态。

一次访存必须依次回答四个问题

1. 这个虚拟地址属于当前地址空间吗？
2. 本次访问类型（R/W/X）被允许吗？
3. 对应数据当前是否 resident？
4. 如果 resident，它映射到哪个 physical frame？

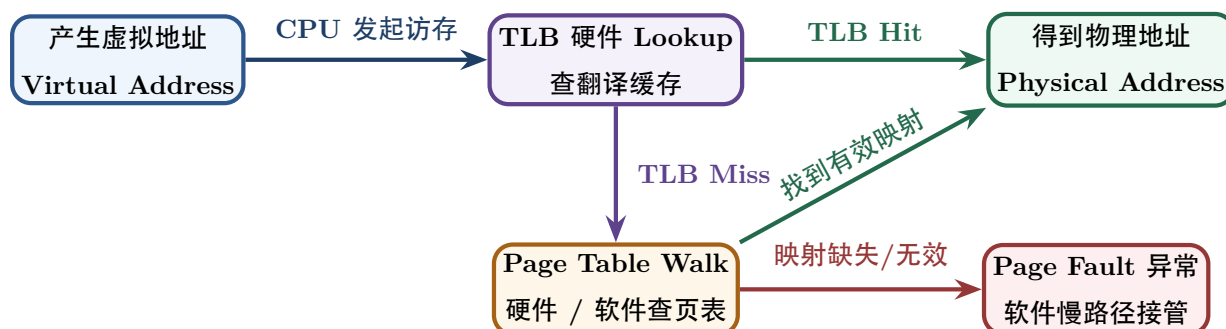
6 TLB：翻译本身也需要缓存

6.1 为什么页表不能每次都完整查

如果每次 load/store 为了得到 PA 都先访问多级页表，那么一次数据访问会附带多次额外内存访问。TLB 用一个小型硬件 cache 保存近期地址翻译：

$$VPN \rightarrow PFN/permission.$$

6.2 快路径与慢路径



6.3 三种 miss/fault 必须彻底分开

现象	缺失的核心内容	是否必然进入 OS 内核?
TLB Miss	硬件地址翻译缓存项 ($VPN \rightarrow PFN$)	不一定；硬件 Page Walk 成功时直接补全 TLB，不触发 OS 异常
Page Fault	当前访问无法沿正常页表路径完成	必须进入 ；需要 OS 捕获异常、分配页框/解装入/COW 或终止
Cache Miss	目标数据 Cache line 不在 L1/L2/L3 Cache	不进入 ；属于存储层次硬件行为，由总线与主存完成加载

典型错误

TLB Miss 不等于 Page Fault；Page Fault 也不等于“页面一定在磁盘上”。COW、lazy allocation、权限错误都可以触发 page fault，而数据可能从未存在于磁盘。

6.4 有效访问时间 EAT 的模型意识

408 常用理想化模型：

$$EAT = h \cdot T_{hit} + (1 - h) \cdot T_{miss}.$$

关键不是背固定式子，而是明确每条路径到底发生几次 TLB 查询、页表内存访问和最终数据访问。题目改变“TLB 与 Cache 是否并行、页表几级、TLB miss 是否硬件 walk”等假设时，公式必须重新从事件路径推导。

工程边界：ASID / PCID

地址空间切换后必须确保旧 TLB 项不会被错误用于新地址空间，但这并不意味着现代 CPU 每次 context switch 都必须“完全清空 TLB”。ASID/PCID 一类标签允许不同地址空间的翻译共存，从而降低切换成本。408 先掌握“一致性必须维护”，再把标签机制作为现代扩展。

EAT 先画路径再代数

若 TLB 命中概率为 h ，TLB 查询耗时为 t ，内存访问为 m ，单级页表且串行查找时，常见口径为

$$EAT = h(t + m) + (1 - h)(t + 2m).$$

若题目把 TLB 查询与首个内存访问并行、或给出多级页表/硬件 page walk，则不得套用此式；逐条数“TLB 查询、页表项访问、最终数据访问”即可恢复正确路径。换页表/地址空间后还要考虑旧 TLB 项的失效或 ASID/PCID 标签。

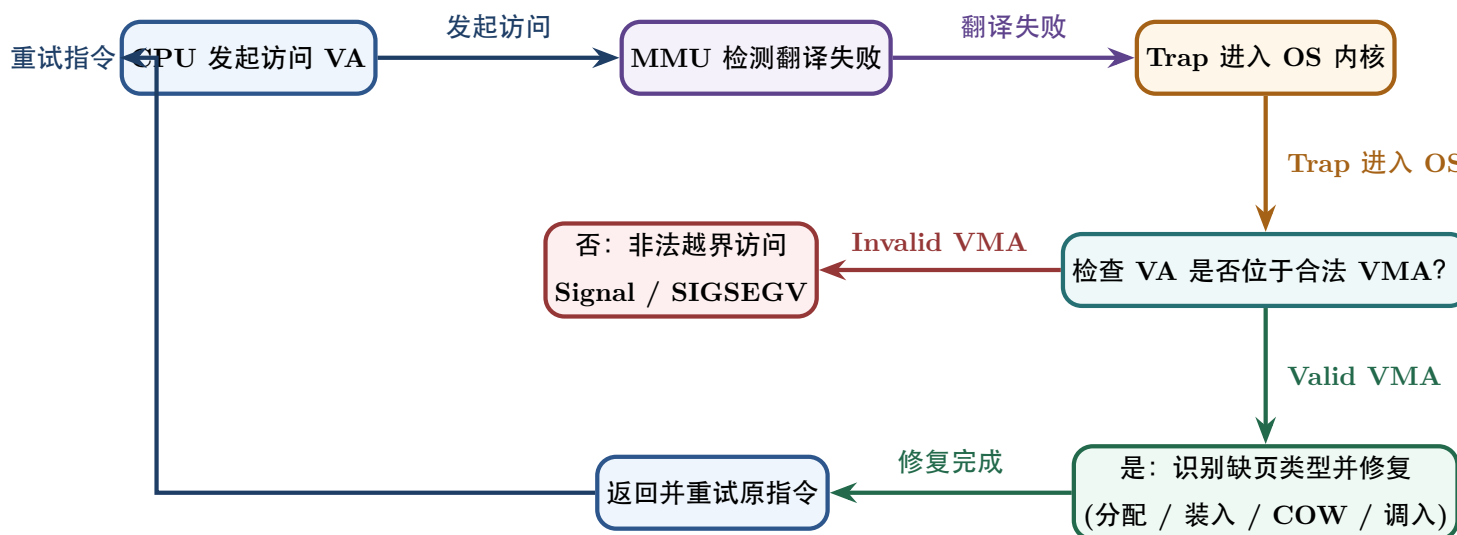
7 Page Fault：虚拟内存的慢路径入口

7.1 定义：不是错误，而是“硬件无法完成当前翻译/访问”

MMU 在正常快路径不能完成访问时触发 page-fault exception。内核收到后必须先分类：

合法但需补工作 or 非法访问

7.2 统一处理流程



缺页处理不是固定的“读盘后结束”，更不能一概写成“缺页 ⇒ 阻塞 ⇒ 调度”。Page Fault 首先是当前访存指令触发的**同步异常入口**：内核保存/读取 fault 现场，验证地址归属与访问权限，再按 fault 原因选择修复或拒绝。若是 demand-zero、COW 等**纯内存内修复**，内核可以在当前 task 的内核路径中分配/复制页、更新 PTE 与必要的翻译状态，然后直接返回并**重试原指令**，期间不要求切换进程；若是 file-backed page-in、swap-in 或 dirty victim write-back 等需要等待设备完成的路径，当前 task 才会进入 Blocked，

设备完成后被唤醒到 Ready，再由调度器决定何时重新 Running。若地址或权限非法且没有合法修复机制，则不应重试，而应报告异常并终止。

7.3 Page Fault 的六种常见原因与处理

Page Fault 类型	硬件/应用触发的具体原因	OS 内核接管后的处理动作
Demand-zero / 延迟分配	地址合法，但应用首次访问，物理页尚未分配	分配空闲 Frame，物理数据清零，更新可用 PTE
File-backed 按需页	代码/数据/mmap 页合法，但磁盘文件内容尚未载入	从文件系统 / 页缓存读取数据，建立页表映射
Swap-in 页面换入	页面曾驻留主存，先前因内存压力被换出到 Swap 区	从 Buddy 分配 Frame，从 Swap 磁盘读回，恢复 PTE
Copy-on-Write (COW)	原本可写的私有页因 fork 等原因被父子临时共享并去掉写权限，某一进程试图写入	先确认该 PTE 的软件状态确实是 COW，而不是“本来就只读”；共享引用仍多于一个时分配并复制 Frame、更新 faulting PTE 为私有可写并减少旧页引用。若实现确认该页只剩当前映射，也可直接恢复写权限作为优化；原本只读代码页的写入不可按 COW 修复
Protection Fault 权限错	页面映射存在，但本次访问类型 (R/W/X) 违规	校验合法机制 (如 COW 修复)；不可修复则报 SIGSEGV
Invalid Address 越界	虚拟地址根本不属于任何合法 VMA 区域	软件无法修复，内核向进程发送 SIGSEGV 信号终止进程

408 必须形成的 Page Fault 观

“缺页”在经典请求分页题中常特指页面不在主存；但从完整 OS 机制看，page fault 是更广义的异常入口。做 408 题时先服从题目采用的经典模型；做系统理解时再区分 demand paging、COW、permission 等不同 fault 原因。尤其要守住两条边界：**Page Fault 不等于页面一定在磁盘**；**Page Fault 也不等于当前进程一定阻塞**。是否阻塞取决于修复是否需要等待外部事件。

8 一个 Page 的一生：从不存在到驻留、变脏、被回收

8.1 Page Lifecycle 是虚拟内存最重要的动态模型

一个匿名 heap page 的生命周期可以写成：

```
VMA exists → PTE absent → first touch fault → resident clean
                → dirty / accessed → reclaim candidate → evicted / nonresident
```

未来再次访问时，再经 fault 恢复 residency。

8.2 Resident 与 Valid 是两个容易混淆的词

不同教材/体系结构对 valid/present 命名并不完全一致，因此考试时要看题目定义。方法上应分别记录：

- 地址是否合法属于进程；
- 页面当前是否驻留 RAM；
- 访问权限是否允许。

不要只看到一个“valid bit”就把三层语义压成一层。

9 Demand Paging：为什么“需要时再做”通常更划算

9.1 Lazy 是一种通用系统策略

立即为所有潜在页面分配 RAM 会浪费：

- 从未访问的 heap 页面；
- fork 后马上 exec 的大量页面；
- 稀疏数组的巨大空洞；
- 可执行文件中实际从未执行/读取的区域。

于是系统把成本延迟到第一次使用：

Reserve now, pay on first touch.

9.2 代价：首次访问被推入慢路径

Lazy allocation 并没有消灭成本，而是把：

allocation + initialization + mapping

从申请时移动到了首次访问时。因此平均性能是否更好，取决于“有多少申请最终真的会使用”。

10 页面置换：RAM 不够时谁应该留下

10.1 先问为什么需要 replacement

只要 resident working sets 的总需求长期逼近/超过可用 frames，就必须回收一些页面。Replacement 是选择 victim 的 policy，而 page fault / page-in / PTE update 是实现机制。

10.2 缺页率 (Page Fault Rate) 的多因素控制模型

学生最容易产生的错误直觉是：“缺页率高，一定是因为置换算法选得不好。”必须建立综合结果模型：

$$\text{Page Fault Rate} = f(\text{Locality}, \text{Frame Allocation}, \text{Page Size}, \text{Replacement Policy}, \text{Working Set})$$

缺页率由 5 个变量共同决定：

1. **程序局部性 (Locality)**: 时间与空间局部性越好, 缺页率越低;
2. **页框配额 (Frame Allocation)**: 分配给进程的物理页框越多, 缺页率越低 (未遇到 Belady 异常时);
3. **页面大小 (Page Size)**: 页大可包含更多数据 (但页内碎片可能增加), 对缺页率有综合影响;
4. **置换算法 (Replacement Policy)**: 越贴合未来访问趋势 (如 OPT/LRU), 缺页率越低;
5. **工作集大小 (Working Set Size)**: 物理页框低于工作集阈值时将发生抖动 (Thrashing), 缺页率急剧上升。

10.3 四个经典策略的思想, 而不是口号

置换算法	核心假设与选择规则	算法关键性质与工程评价
OPT (最佳置换)	淘汰未来最长时间不会被访问的页面	理想极限下界; 需已知未来访存序列, 用于评估性能上界
FIFO (先进先出)	淘汰最早进入内存的页面	简单但盲目; 可能出现 Belady 异常 (框增多缺页反增)
LRU (最近最少使用)	淘汰最长时间未被访问的页面	利用时间局部性; 精确维护全序访问的硬件/软件开销极高
CLOCK (时钟算法)	用 Reference Bit 近似 LRU (Second Chance)	工程可行性极佳; 访问页重置标志, 再给一次机会

为什么明明 LRU 效果好, 现实系统却采用 CLOCK?

精确 LRU 要求系统在**每一次访问内存时**都更新页面访问的时间戳或调整双向链表/硬件栈。在 CPU 频率极高的快路径下, 每次访存维护全序关系的代价过高。

CLOCK / 二次机会算法利用 PTE 中的 Reference Bit, 将“精确的时间全序”退化为“分批重置的访问标志 (Approximate Recency)”, 在牺牲精确 LRU 次序的前提下, 大幅降低每次访存维护全序关系的开销。

CLOCK、NRU 与改进 CLOCK 不要混成同一个名字

三者都利用“近期访问”信息近似未来价值, 但状态机不同:

- **Basic CLOCK / Second Chance**: 主要使用访问位 A/R 和环形指针。扫描到 $A = 1$ 时清零并跳过, 扫描到 $A = 0$ 才选择 victim;
- **NRU (Not Recently Used)**: 通常周期性清访问位, 并依据 $(A/R, D/M)$ 把页面分为若干类别, 优先从“近期未访问”且 clean 的低代价类别选择;
- **Enhanced / Improved CLOCK**: 把 CLOCK 的环形扫描与 dirty/modification 信息结合。不同教材可能规定不同扫描轮次, 因此考试必须按题设给出的具体规则执行。

访问位回答“近期是否被使用”, 脏位回答“淘汰是否可能需要写回”; 二者分别是**未来价值估计**与**置换成本估计**, 不能把 $D/M = 0$ 当成“最近未访问”。

10.4 Dirty Page 为什么影响置换代价

若 victim 是 clean page, 若有可靠后备对象, 通常可以直接丢弃映射并未来重新获得; dirty page 则可能需要 write-back:

$$T_{\text{fault}} \approx T_{\text{victim write}} + T_{\text{page in}} + T_{\text{restart}}.$$

所以题目给 dirty bit 时, 通常在暗示“换出成本不同”。

10.5 Frame Allocation 与 Replacement Scope

还要区分:

- 每个进程分多少 frames: fixed / variable allocation;
- victim 只能从自己页面选还是可从全局选: local / global replacement.

这是“资源配额”与“局部选择策略”两个层次。

页框分配还可沿三条正交轴描述: 固定/可变配额, 局部分配/全局分配, 以及平均分配/按比例或按优先级分配。页面调入来源既可以是请求进程自己的可执行文件/映射文件, 也可以是交换区; 预取 (prepaging) 能利用空间局部性减少后续 fault, 但误取会挤占有效页。页框回收通常优先选择未锁定、非内核关键、可丢弃的 clean file-backed 页; dirty anonymous 或 dirty file-backed 页需先写回, 锁定页和正在 I/O 的页不能作为 victim。

10.6 最小页框数与有效驻留集: 正确性底线不等于性能目标

页框需求至少有两条不同的约束线:

- **指令可完成的最小驻留需求:** 由题设 ISA、指令/操作数可能跨页以及是否允许在一条指令执行中途处理 fault 等语义共同限定; 它回答“能否正确推进”。不能脱离题设背一个固定数字。
- **当前局部性的有效驻留需求:** 由最近访问集合和阶段性工作集决定; 它回答“能否以可接受缺页率推进”, 会随时间变化。

因此“页框不少于硬件底线”只是正确性条件, 不保证性能; “页框接近当前工作集”是性能目标, 也不是永久不变的常数。

10.7 Working Set 与 PFF: 一个预测模型, 一个反馈控制器

工作集用最近窗口内访问过的不同页面近似下一阶段需要保留的集合:

$$W(t, \Delta) = \{\text{在窗口}(t - \Delta, t] \text{ 内被访问的页面}\}.$$

窗口过小会漏掉仍有用的页面, 过大又会把已经离开的局部性阶段保留过久。

Page-Fault Frequency (PFF) 不直接猜页面集合, 而是观察结果并调整配额:

fault rate too high $\Rightarrow$ try more frames/admission control, fault rate very low $\Rightarrow$ frames may be reclaimable.

局部性假设与颠簸的正反馈链 局部性是经验性统计规律而非绝对逻辑保证：随机访问、工作集持续扩张或访问阶段切换都会使窗口模型失准。

抖动 (Thrashing) 具有恶性正反馈机制：

1. 活跃工作集超出可用物理页框，缺页率急剧上升；
2. 进程频繁阻塞等待磁盘 I/O，导致 CPU 利用率断崖式下跌；
3. 长程调度器误判“系统负载不足”，盲目调入新进程加剧多道程序度；
4. 新进程进一步争抢极度匮乏的页框，缺页与阻塞彻底失控！

治理手段的分层解耦 治理 Thrashing 必须作用于不同环节，不能误以为“单靠更换置换算法”就能解决：

- **准入控制 (Admission Control)**：降低多道程序度，挂起/调出部分次要进程；
- **资源供给 (Resource Allocation)**：增加可用物理页框或扩充内存容量；
- **需求估计 (Demand Estimation)**：基于 Working Set 窗口动态预留工作集页框；
- **结果反馈 (Feedback Tuning)**：根据缺页率 (PFF) 阈值动态增减分配给进程的 Frame 数。

工作集提供对象层预测，PFF 提供结果层反馈；两者协同建立颠簸防御闭环。

11 Working Set 与 Thrashing：负载过高时越调度越慢

11.1 局部性为什么支持虚拟内存

程序通常在某一时间窗口只频繁使用较小页面集合，称为工作集的思想基础。只要这些热点页大部分驻留，page fault rate 就可以保持较低。

11.2 Thrashing：页框不足导致反复换入换出

若运行中各进程的活跃工作集总需求长期超过可用页框，系统就有很高的颠簸风险：

$$\sum_i WorkingSet_i > AvailableFrames,$$

系统会不断：

$$Fault \longrightarrow Evict \longrightarrow Run\ a\ little \longrightarrow Fault \longrightarrow Evict.$$

于是大量时间花在移动页面，而不是有用计算。

跨科统一：Thrashing 与网络拥塞、锁竞争属于同一种系统现象

当并发工作量超过稀缺资源承载能力，增加任务不再增加吞吐，反而提高管理和等待成本。解决思路不是“再多塞几个任务”，而是控制 admission、减少工作集、增加资源或改变调度策略。

12 物理内存管理：Buddy 应该放在这里，而不是和 malloc 混在一起

12.1 历史连续分区与 Free-Space Management

First Fit、Next Fit、Best Fit、Worst Fit、Quick Fit 等算法回答的是：

面对多个空闲连续块，选择哪一个满足连续请求？

它们主要研究查找速度、外部碎片、回收合并等权衡。作为 408 历史模型应掌握，但不要把它们直接等同于现代进程虚拟内存的物理页映射。

12.2 Buddy System：管理成组连续物理页

Buddy 以 2^k 页为阶数组织空闲块。请求需要 2^k 时：

1. 若该 order 有空闲块，直接分配；
2. 否则从更高 order 分裂；
3. 释放后若 buddy 同阶且空闲，就递归合并。

若基于按 2^k 对齐的地址表示，伙伴定位常可用：

$$buddy_addr = addr \oplus 2^k$$

(具体公式中的单位必须与 $addr$ 和 2^k 的定义一致)。

工程定位

Linux 内存管理把 Page Allocation、Page Tables、Process Addresses、Page Reclaim、Swap、页缓存等作为相互关联但不同的子系统；Buddy 属于物理页分配层，而不是用户态 `malloc()` 的直接算法。

12.3 为什么还需要 Slab/SLUB 一类对象分配器

Buddy 擅长按页/连续页管理物理内存，但内核经常需要大量小型固定结构 (task、inode、dentry 等)。若每个小对象都独占整页会浪费，于是在页分配器之上再建立对象 cache。这再次证明：

物理页分配 $\neq$ 对象分配

13 共享、Copy-on-Write 与 fork：首次写入时才复制物理页

13.1 fork 的语义承诺

子进程必须看到“创建时与父进程相同”的初始内存内容，但这并不要求立即复制每个物理页。

13.2 COW 的完整状态转换

Fork 前：

$$P : VPN_1 \rightarrow PFN_8 \quad (RW)$$

Fork 后（只对**原本可写的私有页**建立 COW 语义）：

$$P : VPN_1 \rightarrow PFN_8 \quad (RO/COW)$$

$$C : VPN_1 \rightarrow PFN_8 \quad (RO/COW)$$

并让物理页的存活引用计数从 1 增至 2。这里的 ‘COW’ 是内核必须能够辨识的软件状态；**仅仅看到 PTE 的写位为 0，不能推出它是 COW**，因为代码段等页面本来就应只读。

若 child 首次写：

$$Store \rightarrow Protection\ Fault \rightarrow Kernel\ COW\ Handler$$

内核先验证“地址合法 + 本次是写 fault + PTE 确为 COW”。若原页仍被多个有效映射引用，则：

$$Allocate\ PFN_{15} \rightarrow Copy(PFN_8) \rightarrow C : VPN_1 \rightarrow PFN_{15}(RW) \rightarrow Ref(PFN_8) - 1.$$

父进程继续指向原页。若实现能够证明原物理页已经只剩 faulting mapping，一个合法的优化是无需复制、直接恢复该 PTE 的写权限；这不是 COW 语义成立的前提，只是避免无意义复制的工程优化。

COW 保持私有内存语义的方法

COW 不是“共享内存 IPC”，而是：**先共享物理实现，继续向每个进程承诺私有内存语义；只有写入会破坏私有性时才复制**。因而必须同时维护两个不变量：原本只读的页面绝不能因为一次 write fault 被错误升级为可写；一个共享物理页只有在最后一个有效 owner/reference 消失后才能真正回到 free list。引用计数表达的是该 Frame 的**存活责任**，不是简单等于“系统里有几个进程”。

13.3 为什么 fork + exec 特别受益

若 child 很快 exec()，原地址空间会被新程序映像替换；立即复制父进程全部内存的工作大多白费。COW 将这部分复制完全避免。

14 mmap：把文件页直接映射进虚拟地址空间

14.1 先分 anonymous 与 file-backed

- Anonymous mapping: heap/stack/匿名 mmap，内容通常不来自普通文件；首次使用可 demand-zero，内存压力下可涉及 swap。
- File-backed mapping: VMA 关联文件和 offset，页面内容由文件作为后备对象。

14.2 mmap 的核心不是“把整个文件读进内存”

更准确的模型是建立：

$$VirtualPage \leftrightarrow FileOffset$$

关系。首次访问某个尚未驻留的 file-backed page 时：

$$VA \rightarrow PageFault \rightarrow File/PageCache \rightarrow PhysicalFrame \rightarrow PTE.$$

文件映射还要区分私有与共享语义：私有可写映射通常采用 COW，修改不直接回写原文件；共享可写映射允许多个进程观察同一物理页，并在回收时按文件一致性规则写回。file-backed 页的“少一次拷贝”是相对显式 read 的用户缓冲区复制而言，仍可能发生缺页、页缓存查找、页表建立、写回与一致性维护，不能理解成零拷贝或零 I/O。

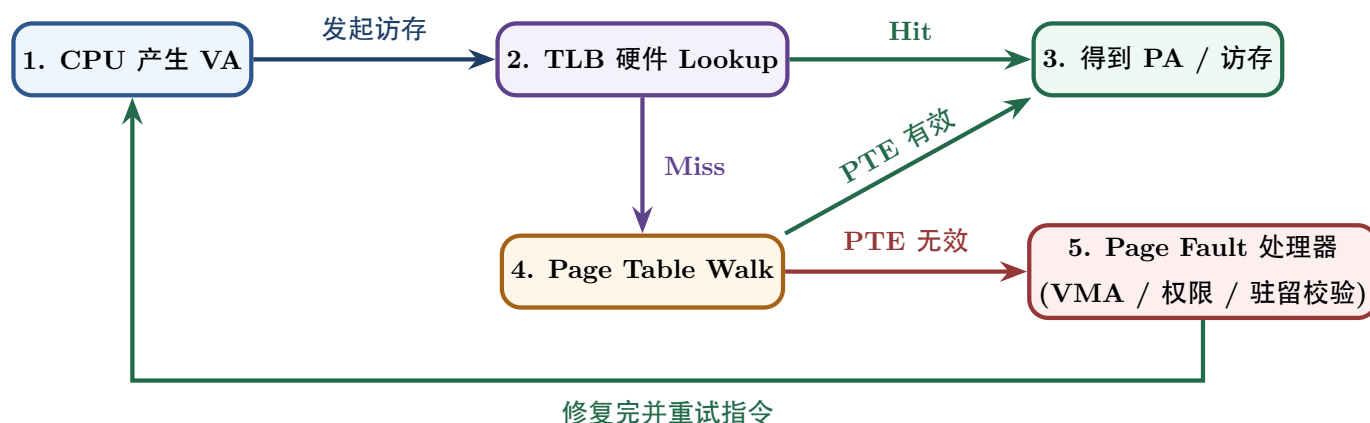
14.3 标准 read 与 mmap 的结构对比

维度 / 特性	标准 read(fd, buf, n) 系统调用	文件映射 file-backed mmap()
应用接口样式	每次数据读取触发显式系统调用	一次映射，之后像常规内存访问一样 load/store
数据内核路径	页缓存 → 显式复制 → 用户缓冲区	文件页对应的物理页可通过页表映射到用户地址空间
数据复制开销	通常需把页缓存中的数据复制到用户缓冲区	可避免这次显式复制，但缺页、页表维护和一致性仍有成本
控制方式	适合流式读写和显式控制读取范围	适合随机访问、共享映射及按页访问文件内容

不要写“标准 I/O 没有 backing store”

文件内容本身的后备对象一直是文件；区别在于 read() 通常把文件数据复制到用户的匿名 buffer，而 file-backed mmap() 直接把虚拟页与文件内容联系起来。讨论 backing 时必须说明“是哪一个页面/对象的 backing”。

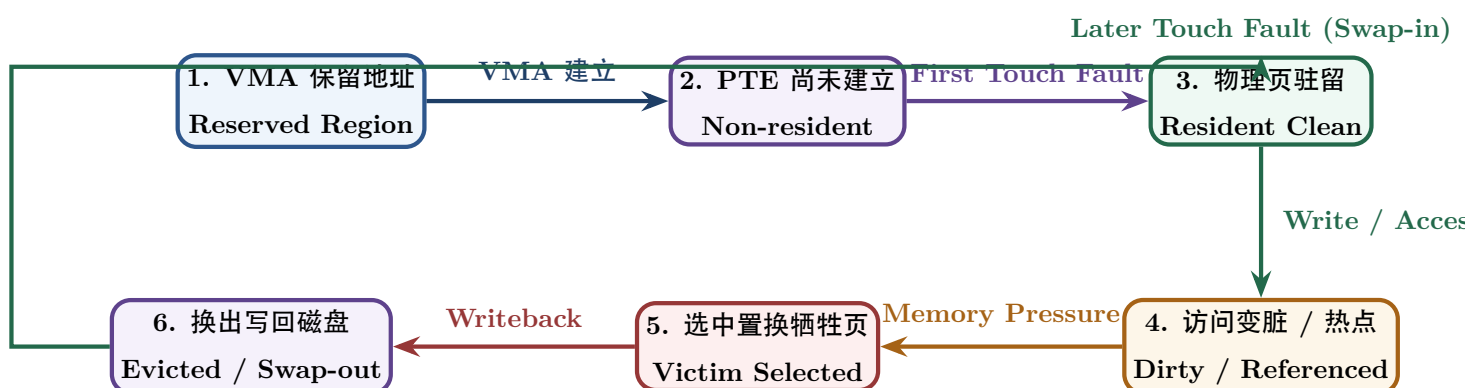
15 综合题一：一次虚拟地址访问



综合题推演时必须记录五类状态

1. 当前地址空间/VMA 是否允许这个 VA;
2. TLB 是否已有 translation;
3. PTE 的 mapping、permission、present 状态;
4. physical frame 是否 resident、clean/dirty、referenced;
5. 若不 resident, backing object 在哪里。

16 综合题二：一个页面的生命周期



这张图把“请求分页、reference/dirty bit、置换、swap/writeback、再次缺页”从多个章节重新还原成一个对象的生命周期。

17 综合题三：fork + COW 的状态变化

手推 COW 固定四步

1. 共享前：谁映射到哪个 PFN?
2. fork 后：父子 PTE 指向同一 frame，但写权限怎样改变？引用计数如何变化？
3. 写 fault：是谁写？MMU 为什么 fault？内核如何判断这是 COW 而不是非法写？
4. 修复后：谁得到新 frame，旧 frame 谁继续引用，权限如何恢复？

18 最小调用接口：先定位地址、映射、驻留还是翻译

遇到虚拟内存问题时，本册只要求先确定当前问题落在哪一层：地址表示、页表映射、物理驻留、硬件翻译、fault 修复或回收策略。若题目开始要求固定计算步骤、页面访问串模拟、题型路由或考场检查顺序，转入同目录训练文档 地址翻译与缺页.md 与 页面驻留与置换.md；若同时追踪 TLB/MMU 与 OS fault handler 的软硬件交接，则转入 X-B02。

第一观察点不是“这像哪种题型”，而是：当前缺失或变化的究竟是哪一层状态？只要这一层没有锁定，就不应继续累加访问次数或套置换算算法。

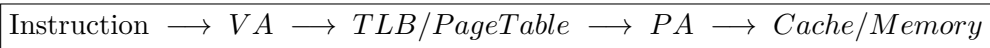
19 教材模型与现代工程：哪些细节要分层记忆

机制主题	408 考研核心教学模型	现代 Linux 内核工程真实实现
页表数据结构	1 / 2 / 多级页表，VPN → PFN 直译	Linux 抽象 5 级页表 (pgd/p4d/pud/pmd/pte)，按架构自动折叠
TLB 一致性	切换地址空间 (切 CR3) 清空 TLB	使用 ASID / PCID 进程上下文标签，避免全量 Flush 带来的高开销
Page Fault 异常	请求分页缺页、访问越界	广泛依赖 Fault 实现 Lazy Alloc, COW, File-backed mmap, KSM 等
页面置换算法	FIFO / LRU / CLOCK / OPT	采用 Active / Inactive 双链表与 Multi-Gen LRU (MGLRU) 估计热度
物理页管理	Buddy 伙伴系统分配阶数页	在 Buddy 之上建立 SLAB / SLUB 对象缓存器，专供小内核对象
文件映射机制	mmap + Page Fault 按需加载	与页缓存、VFS 回写机制与 Direct I/O 深度耦合

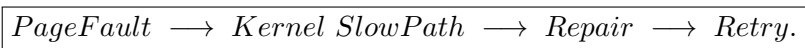
20 跨科桥梁：计组与 OS 在一次访存中怎样接力

软硬件状态所有权		
访存对象 / 动作	CPU / MMU 硬件职责	OS Kernel 软件职责
VA 产生	指令执行单元产生 Effective / Virtual Address	决定进程地址空间布局 (mm_struct) 与允许的 VMA 区域
TLB 缓存	硬件 Lookup 查询并高速缓存 VPN → PFN	在修改页表或销毁进程时负责向 CPU 发送 Flush 命令
Page Table 页表	硬件 MMU 按 ISA 规范自动 Page Walk	物理内存中分配页表页、填写 PTE、配置权限与 Valid 标志
Fault 异常	硬件检测无法完成翻译或权限违规并触发 Exception	捕获异常、分类处理 (Alloc/Disk/COW/Kill)、更新页表与状态
Physical Memory	Cache / Memory Hierarchy 硬件完成物理数据读写	管理全局 Frame 框配额、运行 Page Reclaim 与 Swap 写回

因此完整路径应该始终看成：



若 translation 无法正常完成，才出现：



21 六条核心结论

复原主干

1. 地址是一种名字，不是位置。虚拟内存首先是把程序命名与物理放置解耦。
2. 页表是一层间接映射。多一次查找，换来隔离、共享、保护、稀疏和延迟分配。
3. Allocation、Mapping、Residency、Translation 是四件不同的事。
4. TLB 是 translation cache；Page Fault 是慢路径 entry。
5. 请求分页靠局部性成立；置换是在 RAM 不足时选择谁继续 resident。
6. COW 与 mmap 都是在重新设计“虚拟页背后对应什么对象”。前者延迟私有复制，后者把文件页直接纳入地址空间。

参考资料与工程边界

- Linux Kernel Documentation, *Memory Management Documentation*: <https://www.kernel.org/doc/html/latest/mm/index.html>
- Linux Kernel Documentation, *Process Addresses*: https://cdn.kernel.org/doc/html/latest/mm/process_addr.html
- Linux Kernel Documentation, *Page Tables*: https://cdn.kernel.org/doc/html/latest/mm/page_tables.html
- MIT PDOS, *xv6: a simple, Unix-like teaching operating system* (2025 edition): <https://pdos.csail.mit.edu/6.828/2025/xv6/book-riscv-rev5.pdf>
- OSTEP / CS537 teaching materials: address space, address translation, paging, TLB and swapping chapters: <https://pages.cs.wisc.edu/~remzi/OSTEP/>

与文件系统的接口

这里负责解释文件怎样成为虚拟页的后备对象。路径名、dentry、inode、打开文件描述、文件偏移、索引分配与磁盘布局由文件系统专题定义；两侧通过 `read()` 与 `mmap()` 的数据路径连接。